

Hugging Face Spaces + GitHub + entorno local

Con versiones en Gradio y Streamlit para un clasificador básico de imágenes

Objetivo: Elegir una forma de trabajo según el punto de partida: solo Hugging Face, local, GitHub o Hugging Face Spaces. Cada opción incluye un tutorial paso a paso y archivos mínimos para publicar una app.

Idea central: Hugging Face Spaces ejecuta la app. GitHub puede funcionar como repositorio principal. El entorno local sirve para probar antes de publicar.

Versión preparada para impresión

Índice

- 1. Opciones disponibles y comparación
- 2. Archivos mínimos del proyecto
- 3. Opción mínima: solo Hugging Face Spaces
- 4. Opción Local -> GitHub -> Hugging Face Spaces
- 5. Opción GitHub -> Local -> Hugging Face Spaces
- 6. Opción Hugging Face Spaces -> Local -> GitHub
- 7. Uso diario recomendado
- 8. Errores comunes
- 9. Fuentes consultadas

1. Opciones disponibles y comparación

Hay cuatro formas principales de trabajar con Hugging Face Spaces. La elección depende de si se busca una prueba rápida, un proyecto ordenado, o un flujo profesional con control de versiones.

Por qué existen varias opciones

Hugging Face Spaces puede funcionar como lugar de publicación y ejecución de una app. GitHub puede funcionar como repositorio principal de desarrollo. El entorno local permite probar antes de subir cambios. Por eso, no hay una única ruta correcta: depende del objetivo.

Opción	Cuándo conviene	Qué se usa	Ventaja	Límite
Solo Hugging Face Spaces	Demo rápida, clase, prueba inicial	Navegador + HF Spaces	No requiere Git, GitHub ni instalación local	Menos control y menos orden para proyectos largos
Local -> GitHub -> HF	Proyecto serio, portfolio o entrega	PC + GitHub + HF Spaces	Flujo más prolijo y reproducible	Requiere configurar GitHub Actions
GitHub -> Local -> HF	Ya existe el repo en GitHub	GitHub + PC + HF Spaces	GitHub queda como fuente principal	Hay que crear el Space y conectar token
HF -> Local -> GitHub	Se empezó probando en HF	HF Spaces + PC + GitHub	Permite ordenar después un prototipo	Hay que acomodar remotos y evitar sobrescrituras

Comparación entre Gradio y Streamlit

Framework	Conviene para	Archivo principal	Dato clave en README.md
Gradio	Demos simples de modelos: imagen, texto, audio, clasificación, predicción	app.py	sdk: gradio
Streamlit	Apps con más controles, visualizaciones, paneles, formularios o flujo más tipo dashboard	app.py	sdk: streamlit

Recomendación práctica: Para una primera demo de clasificación de imágenes, Gradio suele ser más directo. Para una app educativa con secciones, controles, gráficos o lógica más extensa, Streamlit puede resultar más cómodo.

2. Archivos mínimos del proyecto

El proyecto mínimo necesita tres archivos: app.py, requirements.txt y README.md. El README.md define cómo Hugging Face Spaces debe ejecutar la app.

Estructura mínima

```
clasificador-imagenes-hf/  
├─ app.py  
├─ requirements.txt  
└─ README.md
```

Versión Gradio: app.py

```
import gradio as gr  
from transformers import pipeline  
  
clasificador = pipeline(  
    task="image-classification",  
    model="julien-c/hotdog-not-hotdog"  
)
```

```
def clasificar_imagen(imagen):
    predicciones = clasificador(imagen)
    resultados = {
        prediccion["label"]: prediccion["score"]
        for prediccion in predicciones
    }
    return resultados

demo = gr.Interface(
    fn=clasificar_imagen,
    inputs=gr.Image(type="pil", label="Subí una imagen"),
    outputs=gr.Label(num_top_classes=2, label="Resultado"),
    title="Clasificador básico de imágenes",
    description="Demo simple con Gradio + Transformers + Hugging Face Spaces."
)

if __name__ == "__main__":
    demo.launch()
```

Versión Gradio: requirements.txt

```
gradio
transformers
torch
pillow
```

Versión Gradio: README.md

```
---
title: Clasificador básico de imágenes - Gradio
emoji: 🖼️
colorFrom: blue
colorTo: green
sdk: gradio
sdk_version: 5.0.0
app_file: app.py
pinned: false
---
```

Clasificador básico de imágenes con Gradio

Demo simple con Gradio y Transformers.

Modelo usado: `julien-c/hotdog-not-hotdog`.

Versión Streamlit: app.py

```
import streamlit as st
from PIL import Image
from transformers import pipeline

@st.cache_resource
def cargar_modelo():
```

```

return pipeline(
    task="image-classification",
    model="julien-c/hotdog-not-hotdog"
)

clasificador = cargar_modelo()

st.title("Clasificador básico de imágenes")
st.write("Demo simple con Streamlit + Transformers + Hugging Face Spaces.")

archivo = st.file_uploader(
    "Subí una imagen",
    type=["jpg", "jpeg", "png"]
)

if archivo is not None:
    imagen = Image.open(archivo).convert("RGB")
    st.image(imagen, caption="Imagen cargada", use_container_width=True)

    predicciones = clasificador(imagen)

    st.subheader("Resultado")
    for prediccion in predicciones:
        etiqueta = prediccion["label"]
        puntaje = prediccion["score"]
        st.write(f'{etiqueta}: {puntaje:.2%}')
        st.progress(float(puntaje))

```

Versión Streamlit: requirements.txt

```

streamlit
transformers
torch
pillow

```

Versión Streamlit: README.md

```

---
title: Clasificador básico de imágenes - Streamlit
emoji: 🐶
colorFrom: blue
colorTo: green
sdk: streamlit
sdk_version: 1.40.0
app_file: app.py
pinned: false
---

```

Clasificador básico de imágenes con Streamlit

Demo simple con Streamlit y Transformers.

Modelo usado: `julien-c/hotdog-not-hotdog`.

Importante: No se recomienda tener al mismo tiempo una app Gradio y una app Streamlit en el mismo Space con el mismo app.py. Para probar ambas variantes, usar dos Spaces diferentes o dos ramas/proyectos separados.

3. Opción mínima: solo Hugging Face Spaces

Esta es la opción más simple. No requiere GitHub, Git local ni instalación de Python. Todo se crea desde el navegador dentro de Hugging Face Spaces.

3.1 Tutorial: solo Hugging Face Spaces con Gradio

1. **Crear un nuevo Space.** Ir a Hugging Face -> Spaces -> Create new Space.
2. **Configurar el Space.** Nombre sugerido: clasificador-imagenes-gradio. SDK: Gradio. Visibilidad: pública o privada.
3. **Crear app.py.** En la pestaña Files, usar Add file -> Create a new file. Nombre: app.py. Pegar el código Gradio de la sección 2.
4. **Crear requirements.txt.** Crear un archivo requirements.txt con gradio, transformers, torch y pillow.
5. **Revisar README.md.** Verificar que tenga sdk: gradio y app_file: app.py.
6. **Guardar cambios.** Cada archivo creado desde la web genera un commit en el Space.
7. **Probar la app.** Ir a la pestaña App. Esperar el build y probar la carga de una imagen.

3.2 Tutorial: solo Hugging Face Spaces con Streamlit

1. **Crear un nuevo Space.** Ir a Hugging Face -> Spaces -> Create new Space.
2. **Configurar el Space.** Nombre sugerido: clasificador-imagenes-streamlit. SDK: Streamlit. Visibilidad: pública o privada.
3. **Crear app.py.** En Files, crear app.py y pegar el código Streamlit de la sección 2.
4. **Crear requirements.txt.** Crear requirements.txt con streamlit, transformers, torch y pillow.
5. **Revisar README.md.** Verificar que tenga sdk: streamlit y app_file: app.py.
6. **Guardar cambios.** Confirmar cada archivo con Commit changes.
7. **Probar la app.** Ir a la pestaña App. Esperar el build y cargar una imagen.

Cuándo elegir esta opción: Es ideal para una clase o prototipo rápido. No es la mejor opción si se necesita trabajar con ramas, probar localmente o mantener un historial más ordenado en GitHub.

4. Opción Local -> GitHub -> Hugging Face Spaces

Esta es la ruta recomendada para un proyecto ordenado. Se desarrolla en la PC, se sube a GitHub y GitHub sincroniza automáticamente con Hugging Face Spaces mediante GitHub Actions.

4.1 Crear el proyecto local

```
mkdir clasificador-imagenes-hf
cd clasificador-imagenes-hf
```

```
python -m venv .venv
.venv\Scripts\activate
```

En Linux o Mac, la activación del entorno virtual sería:

```
source .venv/bin/activate
```

4.2 Elegir Gradio o Streamlit

Para Gradio:

```
pip install gradio transformers torch pillow
```

Para Streamlit:

```
pip install streamlit transformers torch pillow
```

Crear app.py, requirements.txt y README.md usando la variante elegida en la sección 2.

4.3 Probar localmente

Para Gradio:

```
python app.py
```

Para Streamlit:

```
streamlit run app.py
```

4.4 Inicializar Git y subir a GitHub

```
git init
git add .
git commit -m "Primer clasificador de imagenes"
git branch -M main
git remote add origin https://github.com/TU_USUARIO_GITHUB/clasificador-imagenes-hf.git
git push -u origin main
```

4.5 Crear el Space en Hugging Face

1. **Crear Space.** Ir a Hugging Face -> Spaces -> Create new Space.
2. **Elegir SDK.** Si usaste Gradio, elegir Gradio. Si usaste Streamlit, elegir Streamlit.
3. **Usar nombre coherente.** Ejemplo: clasificador-imagenes-hf.

4.6 Crear token de Hugging Face y guardarlo en GitHub

1. **Crear token.** En Hugging Face -> Settings -> Access Tokens. Crear token con permiso write.
2. **Guardar secreto.** En GitHub -> Settings -> Secrets and variables -> Actions -> New repository secret.
3. **Nombre del secreto.** Usar exactamente HF_TOKEN.

4.7 Crear GitHub Action para Gradio

```
name: Sync to Hugging Face Space
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```

jobs:
  sync:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v6

      - name: Sync to Hugging Face Hub
        uses: huggingface/hub-sync@v0.1.0
        with:
          github_repo_id: ${{ github.repository }}
          huggingface_repo_id: TU_USUARIO_HF/clasificador-imagenes-hf
          hf_token: ${{ secrets.HF_TOKEN }}
          repo_type: space
          space_sdk: gradio

```

4.8 Crear GitHub Action para Streamlit

```

name: Sync to Hugging Face Space

on:
  push:
    branches: [main]

jobs:
  sync:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v6

      - name: Sync to Hugging Face Hub
        uses: huggingface/hub-sync@v0.1.0
        with:
          github_repo_id: ${{ github.repository }}
          huggingface_repo_id: TU_USUARIO_HF/clasificador-imagenes-hf
          hf_token: ${{ secrets.HF_TOKEN }}
          repo_type: space
          space_sdk: streamlit

```

4.9 Subir el workflow

```

mkdir .github
mkdir .github\workflows
# crear .github/workflows/sync-to-hf.yml
git add .
git commit -m "Agrega sincronizacion con Hugging Face Spaces"
git push

```

Resultado esperado: Cada push a main en GitHub dispara GitHub Actions. Si el workflow termina correctamente, el Space queda actualizado.

5. Opción GitHub -> Local -> Hugging Face Spaces

Esta ruta sirve si el repositorio ya existe en GitHub. Primero se clona a la PC, luego se agregan los archivos de la app y finalmente se conecta con Hugging Face Spaces.

1. **Clonar el repositorio.** Usar git clone desde la terminal.

```
git clone https://github.com/TU_USUARIO_GITHUB/clasificador-imagenes-hf.git
cd clasificador-imagenes-hf
```

5.1 Agregar archivos de la app

Crear app.py, requirements.txt y README.md usando Gradio o Streamlit según corresponda. Usar los archivos de la sección 2.

5.2 Probar localmente

Para Gradio:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Para Streamlit:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

5.3 Subir cambios a GitHub

```
git add .
git commit -m "Agrega app de clasificacion de imagenes"
git push
```

5.4 Crear Space y token

1. **Crear un Space en Hugging Face.** Elegir SDK Gradio o Streamlit según la versión usada.
2. **Crear HF_TOKEN.** Token de Hugging Face con permiso write.
3. **Guardar HF_TOKEN en GitHub.** Repositorio -> Settings -> Secrets and variables -> Actions.

5.5 Agregar workflow

Crear .github/workflows/sync-to-hf.yml. Usar la versión Gradio o Streamlit de la sección 4.7 u 4.8.

```
git add .
git commit -m "Agrega workflow de sincronizacion"
git push
```

Resultado esperado: GitHub queda como fuente principal. Los cambios locales se suben a GitHub y GitHub actualiza Hugging Face Spaces.

6. Opción Hugging Face Spaces -> Local -> GitHub

Esta ruta sirve cuando se empezó creando un prototipo directamente en Hugging Face y después se quiere ordenar el proyecto con GitHub y una copia local.

6.1 Clonar el Space en local

```
git clone https://huggingface.co/spaces/TU_USUARIO_HF/clasificador-imagenes-hf
cd clasificador-imagenes-hf
```

6.2 Revisar o agregar archivos

Verificar que existan app.py, requirements.txt y README.md. Si falta alguno, crearlo con la variante Gradio o Streamlit de la sección 2.

6.3 Probar localmente

Para Gradio:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Para Streamlit:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

6.4 Subir cambios al Space

```
git add .
git commit -m "Ordena app inicial"
git push
```

6.5 Crear repo vacío en GitHub

1. **Crear repositorio.** En GitHub -> New repository.
2. **No agregar README.** El proyecto ya tiene archivos desde Hugging Face.

6.6 Agregar GitHub como remoto

```
git remote -v
git remote add github https://github.com/TU_USUARIO_GITHUB/clasificador-imagenes-hf.git
git branch -M main
git push -u github main
```

6.7 Ordenar remotos

```
git remote rename origin huggingface
git remote rename github origin
git remote -v
```

Resultado esperado:

```
origin    https://github.com/TU_USUARIO_GITHUB/clasificador-imagenes-hf.git
huggingface https://huggingface.co/spaces/TU_USUARIO_HF/clasificador-imagenes-hf
```

6.8 Agregar GitHub Actions

Crear `.github/workflows/sync-to-hf.yml` y usar el workflow Gradio o Streamlit de la sección 4.7 u 4.8.

```
git add .
git commit -m "Agrega sincronizacion con Hugging Face"
git push origin main
```

Cuidado: Una vez que GitHub Actions queda activo, evitar editar archivos directamente en Hugging Face Spaces. Si se edita en Hugging Face y luego se hace push desde GitHub, GitHub puede sobrescribir esos cambios.

7. Uso diario recomendado

Una vez configurado el flujo Local -> GitHub -> Hugging Face, el trabajo diario queda así:

```
git pull
# modificar app.py, requirements.txt o README.md
# probar localmente
git add .
git commit -m "Mejora la app"
git push
```

Comando de prueba según framework:

```
# Gradio
python app.py

# Streamlit
streamlit run app.py
```

8. Errores comunes

Problema	Causa probable	Qué revisar
El Space no se actualiza	Falló GitHub Actions	GitHub -> Actions. Revisar HF_TOKEN, nombre del repo y permisos.
El workflow falla por permisos	Token sin permiso write	Crear token nuevo con permiso de escritura y guardarlo como HF_TOKEN.
Hugging Face no encuentra la app	README.md mal configurado	Revisar app_file: app.py y sdk: gradio o sdk: streamlit.
Streamlit no arranca	requirements incompleto	requirements.txt debe incluir streamlit, transformers, torch y pillow.
Gradio no arranca	requirements incompleto o error en app.py	requirements.txt debe incluir gradio, transformers, torch y pillow.
Se pisan cambios hechos en Hugging Face	GitHub Actions sobrescribió el Space	Elegir GitHub como fuente principal y no editar directo en HF.

Checklist final antes de publicar

- El proyecto tiene app.py.
- El proyecto tiene requirements.txt.
- El README.md tiene sdk: gradio o sdk: streamlit.
- El README.md tiene app_file: app.py.
- La app funciona localmente, si se usa entorno local.
- El Space está creado con el SDK correcto.
- HF_TOKEN está guardado en GitHub si se usa sincronización.
- GitHub Actions termina sin error.

9. Fuentes consultadas

- **Hugging Face Docs - Gradio Spaces:** <https://huggingface.co/docs/hub/en/spaces-sdks-gradio>
- **Hugging Face Docs - Streamlit Spaces:** <https://huggingface.co/docs/hub/en/spaces-sdks-streamlit>
- **Hugging Face Docs - Managing Spaces with GitHub Actions:** <https://huggingface.co/docs/hub/spaces-github-actions>
- **Hugging Face Docs - GitHub Actions:** <https://huggingface.co/docs/hub/en/repositories-github-actions>
- **Hugging Face Docs - Spaces Configuration Reference:** <https://huggingface.co/docs/hub/spaces-config-reference>
- **Hugging Face Docs - Upload files to the Hub:** https://huggingface.co/docs/huggingface_hub/guides/upload